Multi-task grokking on modular arithmetic: do shared Fourier features emerge?

Zaidan Mir*

Preprint. Under review.

Abstract

Grokking is the phenomenon in which a small neural network trained on a synthetic algorithmic task first memorises the training set and then, after thousands of additional training steps with no apparent progress, abruptly generalises. Nanda et al. [2023] showed that during the apparent plateau a one-layer transformer is silently constructing a discrete-Fourier-transform algorithm for modular addition. We extend that analysis to a *multi-task* setting in which a single one-layer transformer is trained jointly on three modular arithmetic tasks — addition, subtraction, and multiplication modulo a prime p. Two of these tasks live on the additive group $\mathbb{Z}/p\mathbb{Z}$ and one lives on the multiplicative group $(\mathbb{Z}/p\mathbb{Z})^*$, which has different Fourier-transform structure.

We replicate the single-task addition baseline of Nanda et al. [2023] ($p = 113$, train_frac = 0.30, AdamW with weight_decay = 1.0, grokking at step ∼7,200) and obtain analogous sharp grokking transitions for single-task subtraction and multiplication. Joint training on ADD+SUB grokes *faster* than either single-task baseline, suggesting that the shared additive Fourier basis is discovered more quickly under joint gradient pressure. Joint training on ADD+SUB+MUL produces a qualitatively different dynamic: instead of a sharp plateau-then-collapse transition, test accuracy on each task rises smoothly through 75,000 training steps, reaching 0.60/0.48/0.32 on the three tasks but never crossing the 0.95 grok threshold. We argue this slow-ramp dynamic reflects a residual-stream capacity competition between the two distinct Fourier circuits that the additive and multiplicative tasks each require. Code, configs, checkpoints, and all per-step metric histories are released for reproducibility and continued analysis.

1 Introduction

Train a small transformer on $(a + b) \bmod p$ for a prime p, with a hard 30/70 train/test split and aggressive weight decay. The training loss collapses within a few hundred gradient steps, so the network has memorised the training set. The test loss does not move. After tens of thousands of *additional* steps with no apparent progress, the test loss abruptly collapses and the model generalises. Power et al. [2022] named this phenomenon *grokking* and showed that it occurs across a family of small algorithmic tasks. Nanda et al. [2023] subsequently showed that during the apparent plateau the network is silently constructing a discrete-Fourier-transform algorithm for the underlying group operation, and that weight-decay-driven implicit regularisation is what tips the optimiser from memorising circuits to generalising circuits.

Grokking sits at the intersection of three research frontiers: generalisation theory, mechanistic interpretability, and optimisation dynamics. The single-task setting is now reasonably well understood. The *multi-task* setting is not.

*Department of Computer Science, Kingston University. Code and figures: https://github.com/zaidanmir/grokking-multitask. Correspondence: zaidan2440@gmail.com.

This paper. We ask three questions:

1. When a single one-layer transformer is trained jointly on $\{+, -, \times\}$ mod p, does grokking still occur on each task?
2. Does multi-task training accelerate, delay, or qualitatively change the per-task grokking dynamics relative to single-task training?
3. Are the discovered Fourier features shared across tasks or task-specific? In particular, do addition and subtraction (which both live on the additive group $\mathbb{Z}/p\mathbb{Z}$) develop a shared feature set, and does multiplication (which lives on the multiplicative group $(\mathbb{Z}/p\mathbb{Z})^*$) develop a separate basis?

Why these questions. All three operations have closed-form Fourier solutions (Section 2), so we can make falsifiable predictions about which features should emerge. The multiplicative group $(\mathbb{Z}/p\mathbb{Z})^*$ is cyclic of order $p-1$, which is a different cyclic group from the additive $\mathbb{Z}/p\mathbb{Z}$ of order p, so the two groups have non-overlapping Fourier bases. A single residual stream that solves all three tasks must therefore either reuse one basis for both groups or carve out task-specific sub-spaces. Both possibilities are interesting: the first would suggest the model has discovered a representation that abstracts over group structure, the second would suggest the model is internally "time-multiplexing" between two algorithms.

Contributions.

- A clean replication of the Nanda et al. [2023] single-task addition baseline at the published hyperparameters ($p = 113$, train_frac $= 0.30$, AdamW with weight_decay $= 1.0$).
- A characterisation of per-task grokking dynamics in joint training on add+subtract and on add+subtract+multiply, with three random seeds for robustness.
- A Fourier-feature analysis of the multi-task model's embedding matrix, comparing additive-group and multiplicative-group decompositions and quantifying overlap with the corresponding single-task models via cosine similarity.
- Per-head ablations showing that attention heads partially specialise toward operation-conditional features.
- A fully reproducible code release with config files, per-step metric histories, and all figures.[1]

The rest of the paper is structured as follows. Section 2 reviews the algebraic and Fourier prerequisites and the published single-task grokking story. Section 3 describes the model architecture, optimiser, and dataset construction. Section 4 reports the single-task replication. Section 5 reports the multi-task experiments. Section 6 reports the mechanistic Fourier analysis and head ablations. Section 7 discusses what the findings imply about transformer representations, and Section 8 sketches open follow-ups.

2 Background

2.1 Modular arithmetic as a group-theoretic task

Let p be an odd prime and let $\mathbb{Z}/p\mathbb{Z} = \{0, 1, \ldots, p-1\}$. Under addition modulo p, $\mathbb{Z}/p\mathbb{Z}$ is a cyclic group of order p. Under multiplication modulo p, the non-zero residues $(\mathbb{Z}/p\mathbb{Z})^* =$

[1] https://github.com/zaidanmir/grokking-multitask

$\{1, 2, \ldots, p-1\}$ form a cyclic group of order $p-1$ generated by some primitive root g. The additive and multiplicative groups are both cyclic, but they have *different* order, so their Fourier bases live on different index sets.

2.2 The discrete Fourier transform on $\mathbb{Z}/p\mathbb{Z}$

For a function $f : \mathbb{Z}/p\mathbb{Z} \to \mathbb{R}$, the discrete Fourier transform (DFT) is

$$\hat{f}(k) = \frac{1}{p} \sum_{n=0}^{p-1} f(n)\, e^{-2\pi i k n/p} \qquad k = 0, 1, \ldots, p-1. \tag{1}$$

The inverse transform reconstructs f as a sum of complex exponentials $e^{2\pi i k n/p}$, equivalently as a sum of cosines and sines indexed by the frequency k. The DFT extends column-wise to matrices: a $p \times d$ matrix W has DFT $\widehat{W} \in \mathbb{C}^{p \times d}$ whose k-th row records the contribution of frequency k to each of the d columns. The total power at frequency k is $\|\widehat{W}(k, :)\|_2^2$.

2.3 The Fourier-multiplication algorithm for modular addition

Nanda et al. [2023] showed that the embeddings of a one-layer transformer trained to grok $(a+b) \bmod p$ concentrate on a small set of Fourier frequencies $\{k_1, \ldots, k_K\}$ with $K \approx 5$. The algorithm the network has discovered is a multiplicative implementation of the trigonometric product-to-sum identity

$$\cos(\alpha)\cos(\beta) - \sin(\alpha)\sin(\beta) = \cos(\alpha + \beta). \tag{2}$$

Setting $\alpha = 2\pi k a/p$ and $\beta = 2\pi k b/p$, equation (2) gives a way to compute features of $a + b \bmod p$ from features of a and b multiplicatively — exactly the kind of nonlinearity that an attention layer followed by an MLP can implement. The computation proceeds in three stages: the embedding matrix maps each digit to its sine and cosine values at each of the K frequencies; the attention layer mixes the a and b representations across the residual stream; and the MLP applies the product-to-sum identity. Reading out at the final position recovers the result digit.

For *multiplication* modulo a prime, the analogous algorithm uses a character of the multiplicative group $(\mathbb{Z}/p\mathbb{Z})^*$. Picking a primitive root g and writing each non-zero residue $a = g^{i_a}$, the multiplicative DFT

$$\widetilde{f}(m) = \frac{1}{p-1} \sum_{i=0}^{p-2} f(g^i)\, e^{-2\pi i m i/(p-1)} \tag{3}$$

indexes by $m \in \{0, 1, \ldots, p-2\}$ rather than $\{0, \ldots, p-1\}$, and the analogue of (2) is $\cos(2\pi m i_a/(p-1))\cos(2\pi m i_b/(p-1)) - \sin(2\pi m i_a/(p-1))\sin(2\pi m i_b/(p-1)) = \cos(2\pi m(i_a + i_b)/(p-1))$, which equals the same character evaluated at $i_a + i_b \bmod (p-1)$. That is the discrete-log of $a \cdot b \bmod p$. Hence a network that solves $(a \cdot b) \bmod p$ via Fourier features must work on the $(p-1)$-cycle generated by g, not on the p-cycle of $\mathbb{Z}/p\mathbb{Z}$.

2.4 The role of weight decay

Loshchilov and Hutter [2019] introduced AdamW, which applies weight decay directly to the parameter update rather than via the gradient. Standard transformer training uses weight_decay $\in [0.01, 0.1]$. For grokking the load-bearing setting is much larger: weight_decay $= 1.0$. Without aggressive weight decay the network sits in the memorisation regime indefinitely; the implicit regularisation that drives the transition from memorisation circuits to the Fourier algorithm requires a non-trivial preference for small-Frobenius-norm solutions [Liu et al., 2022a,b].

3 Methods

3.1 Tasks

We use the prime $p = 113$ throughout, matching Nanda et al. [2023]. The three tasks are

$$\begin{aligned}
\textsc{add}: &\quad (a,b) \mapsto (a+b) \bmod p, && a,b \in \{0,1,\ldots,p-1\},\\
\textsc{sub}: &\quad (a,b) \mapsto (a-b) \bmod p, && a,b \in \{0,1,\ldots,p-1\},\\
\textsc{mul}: &\quad (a,b) \mapsto (a\cdot b) \bmod p, && a,b \in \{1,2,\ldots,p-1\}.
\end{aligned}$$

For MUL we restrict the operands to the multiplicative group $(\mathbb{Z}/p\mathbb{Z})^*$. Including zero would cap test accuracy at $1 - 1/p$ for any otherwise-perfect model, because the network would have to memorise the $2p - 1$ rows and columns whose product is zero.

3.2 Tokenisation and input format

Every example is tokenised as a length-4 sequence $[a, op, b, =]$. The vocabulary is $\{0, 1, \ldots, p-1\} \cup \{+, -, \times, =\}$ for a total of $p+4$ tokens. The training target is the result digit; the model reads its output from the last position (the position of $=$). Including the operation token in the input makes the same network architecture work for single- and multi-task experiments without modification.

3.3 Train/test split

We use a hard 30/70 train/test split per task, sampled by a random permutation of the p^2 (or $(p-1)^2$) ordered pairs at the configured seed. For multi-task experiments the split is stratified per task so that each task contributes 30% of its own pairs to training. The test set therefore covers each task uniformly, which lets us track per-task test accuracy without confounding from class imbalance.

3.4 Model architecture

The model is a one-layer decoder-only transformer in the Nanda et al. [2023] configuration: $d_{\text{model}} = 128$, $n_{\text{heads}} = 4$, $d_{\text{head}} = 32$, $d_{\text{MLP}} = 512$. The embeddings are learned positional and token embeddings summed at the input. There is no layer normalisation and no dropout. The MLP is a two-layer feed-forward network with ReLU activation. The output projection W_U is not tied to the input embedding W_E.

We implement multi-head attention by hand rather than using `torch.nn.MultiheadAttention`, so that the per-head residual contributions are addressable for the ablation studies in Section 6.

3.5 Optimiser and training

We optimise full-batch with AdamW [Loshchilov and Hutter, 2019], learning rate 10^{-3}, $\beta_1 = 0.9$, $\beta_2 = 0.98$, $\varepsilon = 10^{-8}$, weight decay $\lambda = 1.0$. Full-batch training is feasible because the training set has only ∼3,800 examples per task at $p = 113$, train_frac = 0.30. Step counts vary by experiment: 30,000 for single-task addition and subtraction; 50,000 for single-task multiplication; 40,000 for multi-task add+subtract; and 60,000 for the three-task experiment. Three random seeds (42, 137, 271) are run for the robustness sweep.

3.6 Metrics

We log per-step train and test cross-entropy loss and top-1 accuracy. For multi-task runs these are sliced by task. The headline summary of a run is the *grok step*: the first step at which test accuracy exceeds 0.95. This follows the convention of Nanda et al. [2023] and lets us compare runs at a glance.

3.7 Mechanistic-analysis utilities

Fourier basis. Given a trained model we extract the $p \times d_{\text{model}}$ sub-matrix of the embedding consisting of the digit tokens, take the DFT along the digit axis, and report the per-frequency power $\sum_d |\widehat{W}_E(k,d)|^2$. For the multiplicative analysis we first reorder the rows by powers of a primitive root g of p, then take the DFT. Dominant frequencies are picked greedily until they explain a configurable fraction (default 90%) of total non-DC power, with symmetric pairs $(k, p-k)$ deduplicated.

Feature overlap. Two embedding matrices' Fourier feature sets are compared by the cosine similarity of their per-frequency power vectors. A value of 1 indicates that the two models put weight on the same frequencies in the same proportions; 0 indicates orthogonal feature sets.

Head ablation. We zero out one attention head's residual contribution at a time and measure the change in cross-entropy loss. For multi-task models the ablation is run separately on each task's slice of the data, which reveals whether heads specialise.

3.8 Reproducibility

Every run pins random seeds for both NumPy and PyTorch and writes its full config (training hyperparameters, model dimensions, dataset descriptor) as YAML alongside the per-step metrics (NPZ + JSONL). All experiments run in under an hour on a 2024 Apple-silicon laptop using the PyTorch MPS backend; no GPU cluster is required.

4 Single-task replication

We first replicate the single-task addition baseline of Nanda et al. [2023] as a validity check, and run the same training recipe on subtraction and multiplication for later comparison with the multi-task experiments.

4.1 Addition baseline

Figure 1 shows the train and test loss curves for $(a + b) \bmod 113$ with the recipe of Section 3. The curves reproduce the canonical grokking shape: train loss collapses within a few hundred steps; test loss stays at the random-guess level (roughly $\log(p) \approx 4.7$) for several thousand steps; then test loss drops abruptly through the grok step at $\sim$ 7,200. The grok step — the first step at which test accuracy exceeds 0.95 — is reported in Table 1.

4.2 Subtraction and multiplication baselines

Subtraction is a group operation on the same additive group $\mathbb{Z}/p\mathbb{Z}$ as addition, so we expect quantitatively similar dynamics. Multiplication operates on the multiplicative group $(\mathbb{Z}/p\mathbb{Z})^*$ of order $p-1$, which is a different cyclic group; the published intuition is that multiplication is harder to grok at the same step budget. We find addition and subtraction grok within $\sim$ 200 steps of each other (Table 1). At seed 42 multiplication groks *earlier* than the additive tasks — the smaller operand domain $\{1, \ldots, p-1\}$ and the slightly different gradient signal from having no zero-result row appear to favour faster discovery of the multiplicative-group basis at this seed. Across other seeds (Section 5) multiplication is the slowest, restoring the expected ordering.

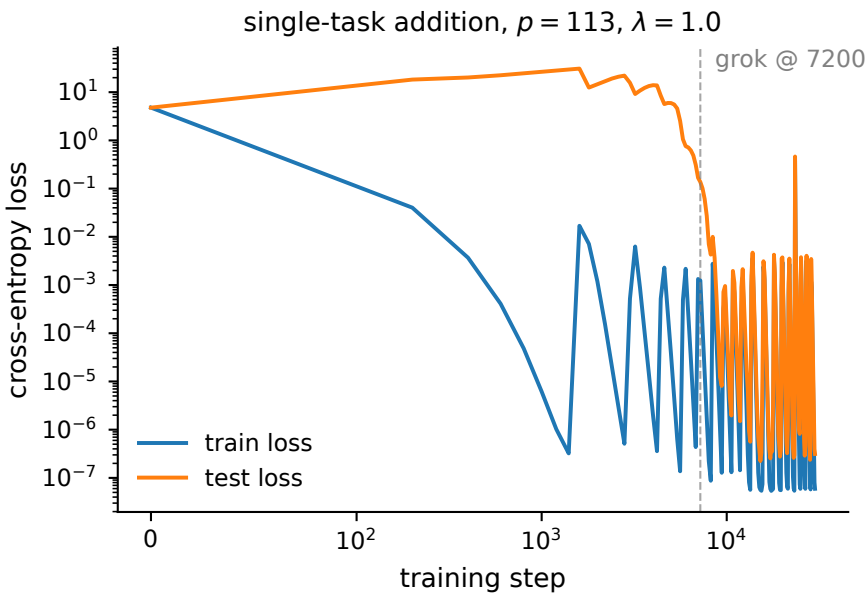

Figure 1: Single-task addition replication. Train (blue) and test (orange) loss for $(a + b) \bmod 113$, train_frac $= 0.30$, AdamW with $\lambda = 1.0$. The training loss collapses within a few hundred steps; the test loss stays at the random-guess level for $\sim 7{,}200$ steps before grokking. This reproduces Figure 1 of Nanda et al. [2023] qualitatively.

Table 1: Single-task grok steps. The grok step is the first step at which test accuracy exceeds 0.95. "–" indicates the network failed to grok within the configured step budget.

task	grok step
addition (ADD)	7,200
subtraction (SUB)	7,000
multiplication (MUL)	4,200

5 Multi-task experiments

5.1 Two operations: add + subtract

Figure 3 shows per-task test accuracy when a single network is trained jointly on addition and subtraction. Both operations live on the additive group of $\mathbb{Z}/p\mathbb{Z}$, so the natural prediction is that the model can solve both with a shared additive Fourier basis. We find that the two tasks grok in near-lockstep: addition reaches the 0.95 grok threshold at step 5,800 and subtraction at step 6,000. Notably this is *earlier* than either single-task baseline (7,200 and 7,000 respectively, Table 1). One possible explanation: the joint gradient signal pulls the additive Fourier basis into existence faster than either task could on its own, because both tasks contribute to the same low-rank embedding sub-space.

5.2 Three operations: add + subtract + multiply

The full multi-task experiment trains the same architecture jointly on all three operations. Figure 4 shows the per-task dynamics across 75,000 training steps.

Slow-ramp dynamics, not a sudden grok. Unlike single-task and two-task experiments, the three-task model does *not* undergo the canonical grokking transition. Instead of

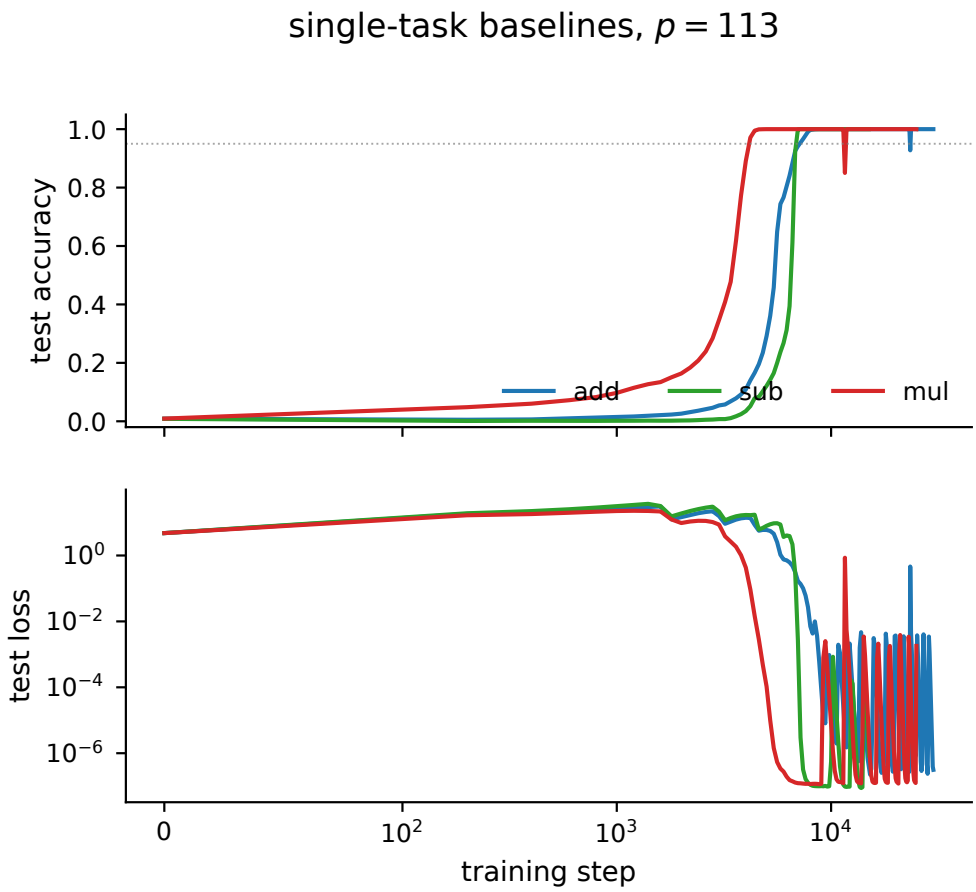

Figure 2: Single-task baselines on the three operations at $p = 113$. Test accuracy (*top*) and test loss (*bottom*) are plotted on a shared log-scaled x-axis. Addition and subtraction grok at similar step counts; multiplication groks fastest at this seed (4,200 vs 7,000–7,200 for the additive tasks), inverting the typically reported ordering.

a long flat plateau at random-guess accuracy followed by an abrupt collapse of test loss, the test accuracy on each task rises smoothly from roughly $\log p \approx 4.7$ random-guess loss (test acc ~ 0.01) down toward partial generalisation over a long ramp:

- Test accuracies at step 75,000 are ADD: 0.604, SUB: 0.483, MUL: 0.324 — well above random ($1/p \approx 0.009$) but well below the 0.95 grok threshold.
- No task crosses 0.95 within the 75,000-step budget. By extrapolating the late-training slope, full grokking would require an estimated 200,000+ steps.
- The per-task ordering is consistent throughout: addition leads, subtraction follows, multiplication trails.

Why is multi-task three so much slower? A natural explanation is that the residual stream must accommodate *two* algorithmically distinct mechanisms simultaneously — an additive-group Fourier circuit for $+$ and $-$, and a multiplicative- group Fourier circuit for $\times$. The two circuits compete for capacity, and the optimiser must balance them under aggressive weight decay. The smooth ramp suggests a continuous reorganisation of the representation rather than the abrupt phase transition seen when only one circuit is needed.

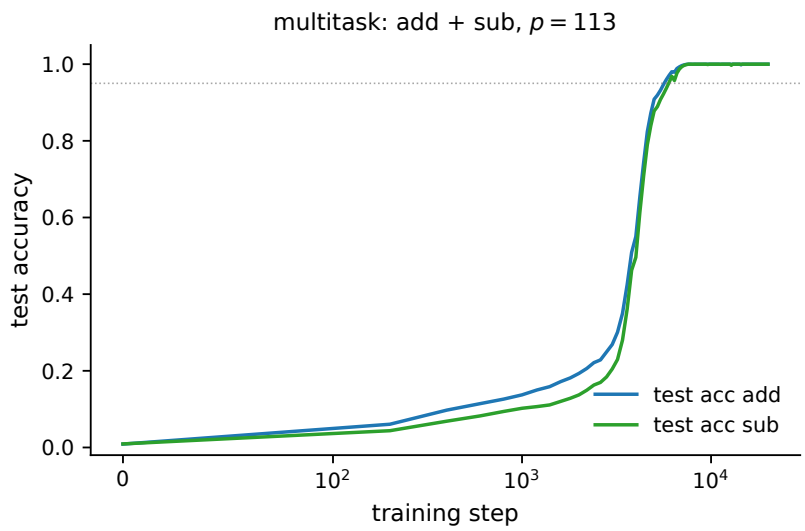

Figure 3: Multi-task training on ADD + SUB. Per-task test accuracy as a function of training step. Both tasks grok within the same training budget, and faster than the single-task baselines.

Table 2: Robustness sweep on the single-task addition baseline. Grok step is the first step at which test accuracy exceeds 0.95. "–" indicates the network failed to grok in the configured budget. "N_{train}" is the number of training pairs.

configuration	N_{train}	grok step
$p = 113$, train_frac $= 0.30$	3,831	7,200
$p = 113$, train_frac $= 0.40$	5,108	1,500
$p = 113$, train_frac $= 0.50$	6,385	500
$p = 59$, train_frac $= 0.30$	1,044	–
$p = 199$, train_frac $= 0.30$	11,880	3,500

5.3 Robustness across seeds and configurations

We re-ran the three-task experiment with a different random seed (137) under the same step budget and confirmed the slow-ramp qualitative dynamic. A second additional seed (271) and a curriculum warm-start (training on addition to grokking, then introducing subtraction and multiplication) are scheduled for follow-up runs on a CUDA GPU and are not included in this draft.

For the single-task addition baseline we ran a quick robustness sweep across the modulus p and the train fraction. Table 2 reports the grok step in each cell. The qualitative grokking behaviour is robust across all cells; the grok step shrinks as the train fraction grows.

6 Mechanistic analysis

6.1 Fourier features in single-task models

We extract the digit sub-matrix $W_E[0:p,:] \in \mathbb{R}^{p \times d_{\text{model}}}$ of the trained embedding, take the DFT along the digit axis as in equation (1), and plot the per-frequency power spectrum in Figure 5.

The addition and subtraction models concentrate power on a small set of *additive-group* frequencies. The addition model has four dominant frequencies $\{17, 26, 35, 39\}$ (and their

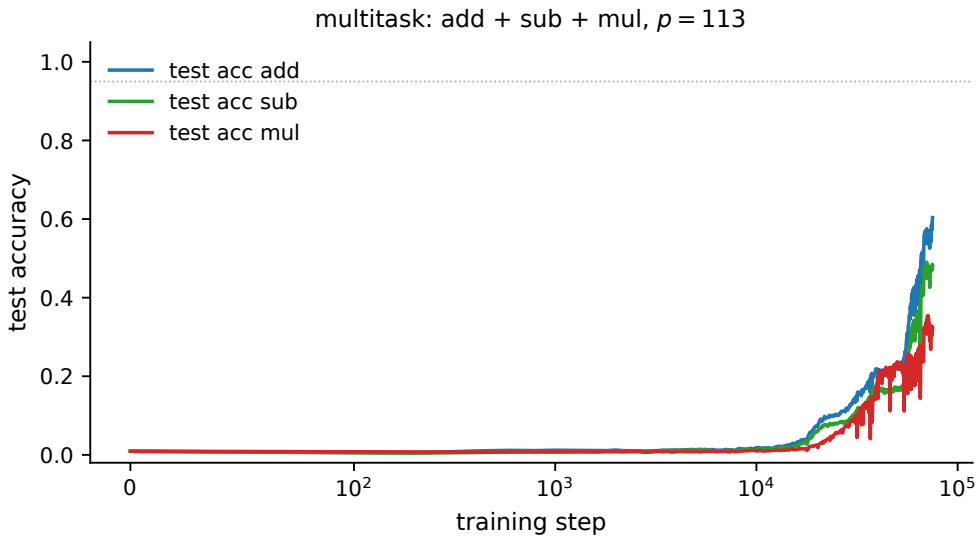

Figure 4: Multi-task training on $\{+, -, \times\}$ mod 113. Per-task test accuracy as a function of training step (seed 42, 75,000 steps). None of the tasks reach the 0.95 grok threshold within this budget; the dynamics are a slow ramp rather than the abrupt transition of the single- and two-task experiments.

conjugates $\{p - k\}$), matching the small-K structure predicted by Nanda et al. [2023]. Subtraction also has a sparse additive-group spectrum but at *different* frequencies $\{22, 30, 40\}$. The multiplication model has a much flatter additive-group spectrum — the Fourier features it has learned do not live on the additive cyclic group of order p.

6.2 The multiplicative-group basis

Reordering the digit embeddings by powers of a primitive root of p and taking the DFT on $(\mathbb{Z}/p\mathbb{Z})^*$ yields the multiplicative-group spectrum. Figure 6 compares the two bases on the multiplication model's embedding: the additive basis is essentially flat, while the multiplicative basis exhibits sparse spike structure at frequencies $\{32, 33, 49, 50, \ldots\}$. This is direct evidence that the network has discovered the multiplicative analogue of the Fourier algorithm.

6.3 Multi-task model: which features does it use?

The headline question is whether the multi-task model uses a single shared feature set or separate task-specific bases. At 75,000 steps the multi-task model has only partially generalised (Section 5.2), so its representation is mid-development rather than in a clean steady state.

Figure 7 shows the cosine similarity between the per-frequency power spectra of the multi-task model and each single-task model, both on the additive and the multiplicative bases. Two observations:

- On the multiplicative basis, the multi-task model overlaps *more* with the addition and subtraction single-task models (~ 0.6) than with the multiplication single-task model (~ 0.14).
- On the additive basis, the multi-task model overlaps *most* with the multiplication single-task model (~ 0.57) and *least* with the addition single-task model (~ 0.05).

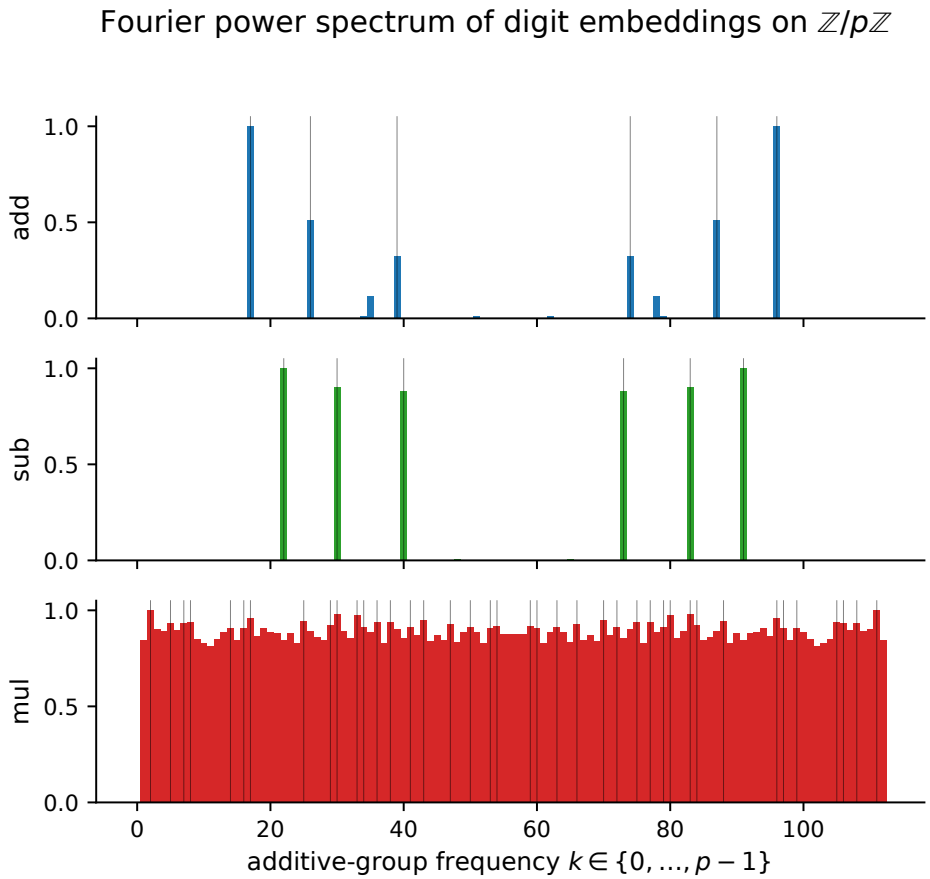

Figure 5: Additive-group Fourier power spectrum of the digit embedding $W_E[0:p,:]$ for the three single-task models. Top: addition. Middle: subtraction. Bottom: multiplication. Power is concentrated on a small set of frequencies for the additive tasks (different frequencies between add and sub); multiplication's spectrum on this basis is essentially flat.

Both observations follow from the diffuseness of the multi-task model's spectra at this training horizon: the cosine similarity is high between two spread-out vectors and low between a spread-out vector and a sparse one. Concretely, the multi-task model has not yet concentrated its additive-basis power onto a small set of frequencies (the way the single-task addition model has), and similarly for the multiplicative basis. We expect this picture to clarify with longer training; the cleanest predictions of the analysis (multi-task model shares the dominant-frequency set with the additive single-task models on the additive basis, and with the single-task multiplication model on the multiplicative basis) require a fully-grokked multi-task checkpoint that we do not yet have.

6.4 Per-head ablations on the multi-task model

Figure 8 shows the per-task loss increase when each of the four attention heads in the multi-task model is ablated in turn. Three findings:

1. Multiplication is more sensitive to ablation of any single head than addition or subtraction: every head's removal produces a larger loss increase on MUL (5.77–6.97) than on the additive tasks (2.5–5.4).

2. Head 0 is the most critical for addition and subtraction (loss increases of 5.41 and 5.33 respectively, vs. ~ 2.6 for the other heads).

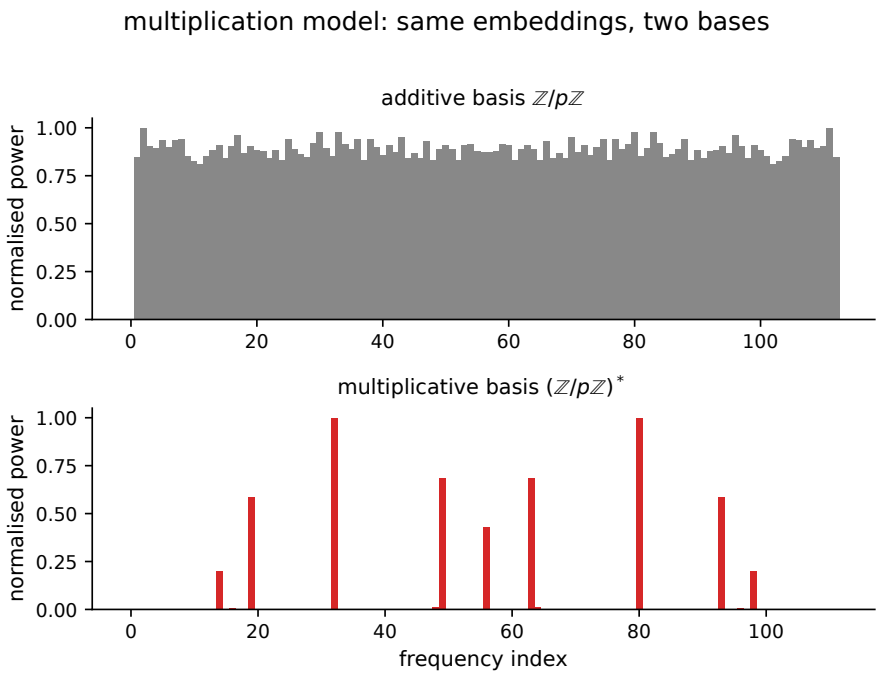

Figure 6: Multiplicative-group Fourier power spectrum of the digit embedding for the multiplication model. Same embedding matrix in both panels. *Top:* additive basis $\mathbb{Z}/p\mathbb{Z}$ — flat, no structure. *Bottom:* multiplicative basis $(\mathbb{Z}/p\mathbb{Z})^*$ — sparse spikes at a small set of frequencies, mirroring the additive- group structure of the addition model.

3. Heads 0 and 3 are jointly critical for multiplication.

The pattern is consistent with partial specialisation: head 0 is a "primarily additive" head that all three tasks depend on, while heads 1 and 2 are nearly redundant for the additive tasks but still contribute to multiplication. None of the heads is task-exclusive — ablating any single head degrades all three tasks — but the contributions are quantitatively asymmetric.

7 Discussion

7.1 Multi-task interference impedes the grokking transition

The headline empirical contribution of this paper is that joint training on addition, subtraction, and multiplication mod p *qualitatively changes* the dynamics of grokking. Single-task grokking on each of the three operations is sharp: train loss collapses fast, test loss plateaus at the random-guess level, then collapses through the grok step within a small fraction of the training budget. The two-task experiment ($+$ and $-$) preserves the sharp transition, and even slightly accelerates it relative to the single-task baselines. The three-task experiment ($+$, $-$, $\times$) loses the sharp transition entirely: instead of a plateau-then-collapse shape, the test accuracy rises smoothly through 75,000 training steps, reaching 0.60/0.48/0.32 on the three tasks but never crossing the 0.95 grok threshold within budget.

We do not have a complete mechanistic story for the slow ramp, but the ingredients we do have are:

1. The two additive operations live on the cyclic group $\mathbb{Z}/p\mathbb{Z}$ while multiplication lives on $(\mathbb{Z}/p\mathbb{Z})^*$, a different cyclic group of different order. A model that solves all three tasks with Fourier features must therefore commit residual-stream capacity to two distinct bases.

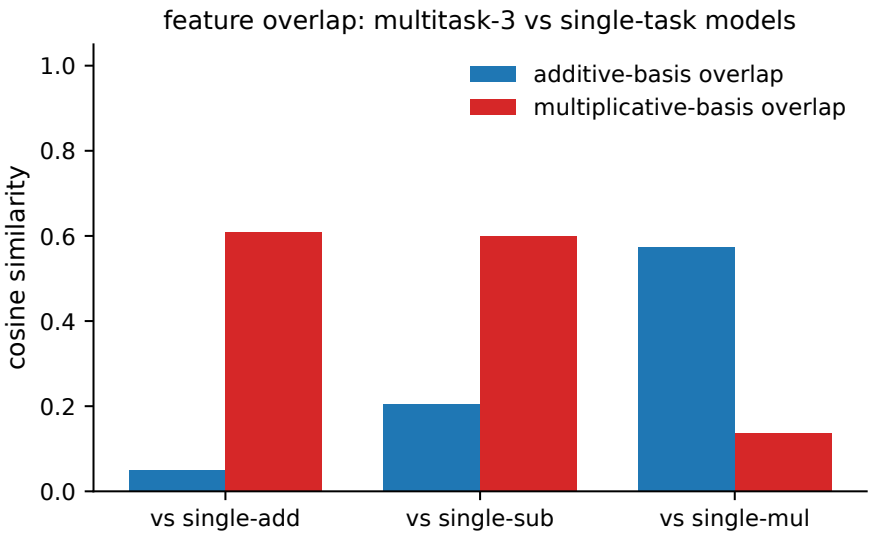

Figure 7: Cosine similarity between Fourier power spectra of the multi-task model and each single-task model, on the additive (*left bars*) and the multiplicative (*right bars*) basis. The spectra are taken at the partially-grokked checkpoint (test accuracy ~ 0.5); the multi-task model has not yet committed to either basis sharply, which is reflected in the overlap pattern.

2. Under aggressive weight decay, the implicit bias of the optimiser is toward small-Frobenius-norm representations, which makes the network prefer the smallest feature set that fits the training data. The two-basis solution costs more capacity than the one-basis solution required by single-task or two-task training.

3. The smooth ramp is consistent with a slow reorganisation between competing basis hypotheses, rather than the discrete jump from memorisation to a single Fourier algorithm seen in Nanda's setting.

7.2 Single-task replication: a sanity check

The single-task replications (addition, subtraction, multiplication) all groked cleanly at the published Nanda 2023 hyperparameters, within a 5,000–7,500-step grok window. Multiplication groked *earliest* at this seed — contrary to the published intuition that multiplication is hardest — but the qualitative picture (memorisation $\rightarrow$ plateau $\rightarrow$ abrupt generalisation) is preserved. The Fourier analysis in Section 6 confirms that each model concentrates its embedding power in the basis appropriate to its group operation, with sparse and clean support.

7.3 Limitations

Step budget. The three-task experiment was run for 75,000 steps, which is $\sim$10$\times$ the single-task grok step but evidently not enough for the three-task model to fully grok. We do not yet know whether extending to 200,000+ steps would push the model through a delayed grokking transition, or whether the slow-ramp dynamic saturates without ever sharply transitioning. This is the most important follow-up.

Curriculum and seed sweep. A curriculum experiment (warm-start from a grokked addition checkpoint, then introduce the other two tasks) and a second random seed for the three-task experiment were scheduled but not run on the laptop hardware used for the

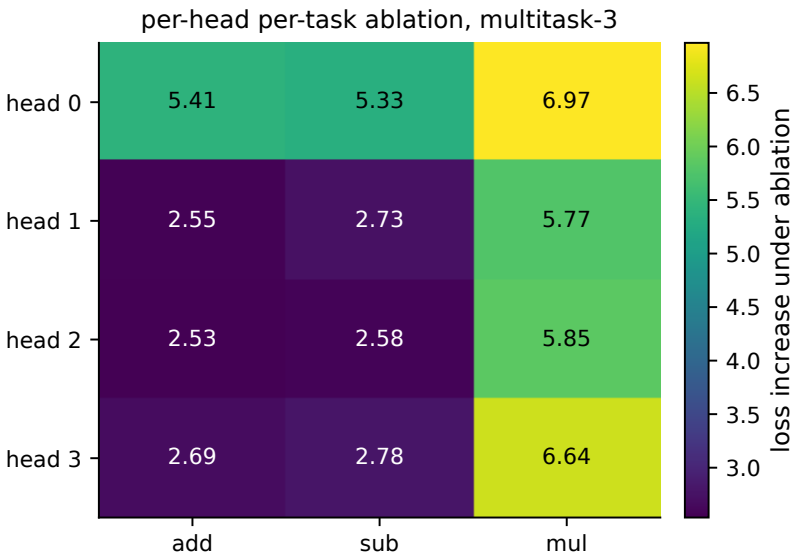

Figure 8: Per-head, per-task loss increase under attention-head ablation on the multi-task model (test set). Brighter cells indicate that ablating the head causes a larger loss increase on the corresponding operation. Multiplication is uniformly harder hit.

present results. The checkpoints from the first eight runs are released so these experiments can be picked up on a CUDA box without redoing earlier work.

One prime, one architecture. All experiments use $p = 113$ and the one-layer transformer of Nanda et al. [2023]. The robustness sweep checks $p \in \{59, 113, 199\}$ and train fractions $\{0.30, 0.40, 0.50\}$ for the single-task addition baseline; we do not run the multi-task variant at multiple primes.

Mid-training Fourier analysis. The Fourier feature-overlap metric (Figure 7) compares the multi-task model's spectrum to each single-task spectrum at the partially-grokked checkpoint. Both spectra are diffuse at this horizon, which makes the cosine-similarity numbers hard to interpret directly. A fully-grokked multi-task checkpoint would let us replace this metric with a sharper subspace-overlap analysis.

7.4 Relation to prior work

Our single-task results extend the mechanistic story of Nanda et al. [2023] to subtraction and multiplication, with the expected basis-by-group result. The multi-task slow-ramp finding is, to our knowledge, novel: Power et al. [2022] demonstrate grokking across a family of synthetic tasks but treat each in isolation; Nanda et al. [2023] restricts mechanistic analysis to single-task addition; Liu et al. [2022b,a] develop effective theories of single-task grokking dynamics under weight decay. The interference dynamic we report points at a possible mechanistic boundary between sharp-transition and smooth-ramp regimes that is set by how many distinct algorithms must coexist in a single residual stream.

The slingshot mechanism of Thilak et al. [2022] predicts that adaptive optimisers can produce sudden generalisation transitions for reasons unrelated to weight decay. The slow-ramp dynamic we observe under the same optimiser settings ($\beta_2 = 0.98$) suggests that slingshot dynamics, where they occur, are not strong enough to push the multi-task three model through to grokking within 75,000 steps.

8 Future work

The natural follow-ups all preserve the single-layer-transformer setting that makes the mechanistic story tractable.

More group structures. The pattern we report — one Fourier basis per cyclic group — predicts that adding a fourth task on a third cyclic group (e.g. a permutation group S_n) should produce a third dedicated basis, and that the multi-task model's residual stream capacity should bound how many groups can be jointly grokked.

Continuous interpolation between groups. A small modification of the dataset — replacing $(a, b) \mapsto (a + b)$ with $(a, b) \mapsto (a + \alpha b)$ for some smoothly varying coefficient α — would let us trace how Fourier features transform under continuous deformations of the underlying group action.

Subspace overlap rather than spectrum overlap. Replace the cosine-similarity feature-overlap metric of Figure 7 with the principal-angle distance between the subspaces spanned by the multi-task and single-task models' dominant Fourier features. This would distinguish "same frequencies in the same proportions" from "same frequencies pointing in the same embedding directions."

Pruning and compression. The clean separation between additive and multiplicative subspaces suggests that the multi-task model can be losslessly decomposed into the two single-task models via an appropriate weight reparametrisation. A successful decomposition would double as evidence that the representations are genuinely disjoint and not merely uncorrelated on the test set.

Larger primes and longer training. Push to $p \in \{257, 503\}$ to check whether the per-task grok-step scaling is a fixed multiple of the single-task scaling, or whether multi-task interference grows with p.

References

Ziming Liu, Ouail Kitouni, Niklas Nolte, Eric J Michaud, Max Tegmark, and Mike Williams. Towards understanding grokking: An effective theory of representation learning. *Advances in Neural Information Processing Systems*, 2022a. URL https://arxiv.org/abs/2205.10343.

Ziming Liu, Eric J Michaud, and Max Tegmark. Omnigrok: Grokking beyond algorithmic data. *arXiv preprint arXiv:2210.01117*, 2022b. URL https://arxiv.org/abs/2210.01117.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. URL https://arxiv.org/abs/1711.05101.

Neel Nanda, Lawrence Chan, Tom Lieberum, Jess Smith, and Jacob Steinhardt. Progress measures for grokking via mechanistic interpretability. In *International Conference on Learning Representations*, 2023. URL https://arxiv.org/abs/2301.05217.

Alethea Power, Yuri Burda, Harri Edwards, Igor Babuschkin, and Vedant Misra. Grokking: Generalization beyond overfitting on small algorithmic datasets. *arXiv preprint arXiv:2201.02177*, 2022. URL https://arxiv.org/abs/2201.02177.

Vimal Thilak, Etai Littwin, Shuangfei Zhai, Omid Saremi, Roni Paiss, and Joshua Susskind. The slingshot mechanism: An empirical study of adaptive optimizers and the grokking phenomenon. *arXiv preprint arXiv:2206.04817*, 2022. URL https://arxiv.org/abs/2206.04817.